PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 01 -

INTRODUÇÃO À IAC: TERRAFORM E OPENTOFU

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 14
O QUE VOCÊ VIU NESTA AULA? 16
REFERÊNCIAS 17

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VEM POR AÍ?

Você já parou para pensar em como a forma de lidar com infraestrutura mudou radicalmente nos últimos anos? Antes, subir um servidor envolvia horas (às vezes dias) de configurações manuais, acessos remotos, algumas horas em DataCenters e CPDs com os Racks sendo nossos “melhores amigos”. Hoje, o cenário é outro: tratamos servidores, redes e até clusters inteiros como linhas de código versionáveis, replicáveis e seguras. É disso que se trata **Infraestrutura como Código (IaC)**.

Nesta primeira aula, conheceremos os fundamentos que sustentam a IaC e entenderemos por que ela se tornou indispensável em times de tecnologia modernos que lidam múltiplos ambientes de alta complexidade. Vamos falar sobre os benefícios, os problemas que podemos resolver e vamos conhecer um pouco das principais ferramentas do mercado, com foco especial em **Terraform** e **OpenTofu**. Você aprenderá a diferença entre as duas e como ambas permitem transformar infraestrutura em algo declarativo e automatizado. Ao final, você já terá colocado a mão na massa, instalando e configurando seu ambiente para provisionar seu primeiro recurso em nuvem ou localmente.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

HANDS ON

Vamos falar sobre o que é **Infraestrutura como Código** e como ela transformou o modo como as equipes de tecnologia constroem e mantêm ambientes. A diferença em relação à abordagem tradicional é clara: em vez de configurar manualmente servidores, redes e serviços na nuvem, descrevemos tudo em código versionado, auditável e facilmente replicável. Isso acelera a criação de ambientes e garante consistência entre ambientes de desenvolvimento, homologação e produção. Ferramentas como **Terraform** e **OpenTofu** são as protagonistas desse movimento.

Antes de avançarmos com exemplos práticos, é importante entender o contexto do OpenTofu. Ele nasceu como um fork do Terraform, depois que mudanças de licenciamento feitas pela HashiCorp geraram discussões na comunidade. A ideia do OpenTofu é preservar o modelo **open source** e garantir que a evolução da ferramenta continue aberta e colaborativa. Em termos práticos, as duas ferramentas funcionam de maneira muito semelhante e, durante esta jornada, você poderá utilizar qualquer uma delas para acompanhar os exemplos.

Na próxima seção, vamos garantir que você tenha o ambiente configurado.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SAIBA MAIS

No Linux, comece instalando os pacotes necessários e depois o Terraform pelo repositório oficial da HashiCorp:

```
    sudo apt-get update && sudo apt-get install -y gnupg
software-properties-common curl
    curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo
apt-key add -
    sudo apt-add-repository "deb [arch=amd64]
https://apt.releases.hashicorp.com $(lsb_release -cs) main"
    sudo apt-get update && sudo apt-get install terraform
    terraform -version
```

Comandos de Instalação Terraform no Linuix
Fonte: Terraform – documentação oficial de instalação (2025)

Para instalar o OpenTofu no Linux, faça o download do binário mais recente e mova para o diretório de execução:

```
    curl -L
https://github.com/opentofu/opentofu/releases/latest/download/
tofu_Linux_x86_64.tar.gz -o tofu.tar.gz
    tar -xzf tofu.tar.gz
    sudo mv tofu /usr/local/bin/
    tofu version
```

Comandos de Instalação OpenTofu no Linuix
Fonte: OpenTofu – documentação oficial de instalação (2025)

No macOS, se você utiliza o Homebrew, a instalação é ainda mais simples. Para instalar o Terraform, rode:

```
    brew install terraform
    terraform -version
```

Comandos de Instalação Terraform no Linuix
Fonte: Terraform – documentação oficial de instalação (2025)

E para instalar o OpenTofu:

```
    brew install opentofu
    terraform -version
```

Comandos de Instalação OpenTofu no Linuix
Fonte: OpenTofu – documentação oficial de instalação (2025)

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Uma vez que as ferramentas estejam instaladas, podemos falar sobre a **estrutura básica de um projeto IaC**. Normalmente começamos com um arquivo chamado `main.tf`, onde definimos os recursos que queremos criar. Podemos ainda usar `variables.tf` para centralizar variáveis, `outputs.tf` para expor valores importantes e um `terraform.tfvars` para armazenar configurações específicas de cada ambiente. Essa estrutura simples já é suficiente para criar um recurso na nuvem e, nas próximas aulas, vamos expandi-la até chegar a arquiteturas completas com múltiplos módulos, loops e condicionais.

Você se lembra da primeira vez que precisou configurar um servidor? Talvez tenha passado horas ajustando permissões, editando arquivos de configuração, conectando via SSH e testando se tudo estava funcionando corretamente. Agora, imagine replicar esse processo dezenas de vezes para diferentes ambientes — desenvolvimento, homologação, produção — e em cada um deles manter a coerência. É justamente desse cenário caótico que nasceu a necessidade da **Infraestrutura como Código**. Ao invés de depender de procedimentos manuais e anotações espalhadas em wikis, passamos a descrever a infraestrutura em arquivos de configuração versionáveis, que podem ser revisados, auditados e replicados com a mesma previsibilidade que temos ao compilar um programa. Essa é a essência da IaC e é por isso que ela se tornou peça fundamental no ecossistema moderno de DevOps e Cloud Computing.

Ao adotar IaC, as organizações não apenas ganham velocidade no provisionamento de recursos, mas principalmente consistência. A mesma linha de código que cria uma rede em um ambiente de testes, é capaz de criar outra idêntica em produção, eliminando as diferenças sutis que antes causavam tantos incidentes e diversas tarefas repetitivas ou penosas para o time de Infra. O benefício mais imediato é a redução do risco humano, mas, com o tempo, o que realmente se percebe é a mudança cultural: equipes passam a tratar a infraestrutura com a mesma disciplina com que tratam o desenvolvimento de software. Isso significa versionar alterações, revisá-las em pull requests, discutir melhorias em código e até mesmo aplicar testes automatizados para garantir que uma mudança de configuração não quebre o ambiente. Essa é uma das bases da cultura DevOps: tudo que se torna penoso ou repetitivo deve ser automatizado para garantir a continuidade e evolução da Infraestrutura. Ao longo das aulas, você verá na prática como isso acontece, mas vale

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

a pena já refletir: **se código se tornou a linguagem universal para software, por que não seria também para infraestrutura?**

Aspecto	Abordagem Tradicional	Infraestrutura como Código (IaC)
Provisionamento	Manual, via consoles e interfaces gráficas.	Automatizado, via arquivos de configuração.
Consistência	Alta chance de erros e diferenças entre ambientes.	Ambientes reproduzidos de forma idêntica.
Escalabilidade	Processo lento e dependente de intervenção humana.	Criação e ajuste de ambientes em minutos.
Versionamento	Mudanças pouco documentadas e difíceis de rastrear.	Código versionado em Git, com histórico completo.
Colaboração	Configurações isoladas entre administradores.	Equipes colaboram no mesmo repositório.
Segurança e Auditoria	Alterações manuais sem trilha clara de auditoria.	Histórico auditável, com revisões e aprovações.
Flexibilidade	Difícil replicar ambientes em diferentes provedores.	Suporte multicloud via provedores.

Tabela 1 – Abordagem tradicional x IaC
Fonte: elaborado pelo autor (2025)

Outro ponto que merece destaque é a diferença entre a abordagem tradicional de gerenciamento de infraestrutura e a proposta da IaC. Assim como foi mostrado na tabela 1, no modelo manual, cada ajuste depende de alguém logando em uma máquina, editando configurações e, muitas vezes, esquecendo de registrar o que foi feito. Isso gera ambientes “frankenstein”, cheios de “adaptações técnicas” ou gambiarras, em que ninguém sabe ao certo qual foi o histórico de alterações.

Na IaC, contudo, cada mudança é declarada em arquivos de texto, revisada em equipe e aplicada de forma automatizada. A diferença é comparável a construir uma casa sem planta versus seguir um projeto arquitetônico detalhado. Usando IaC, você já começa seu projeto com um modelo ou direção a seguir, com padrões bem estruturados e mantendo o histórico de estado da sua infraestrutura. Cada passo é previsível, auditável e replicável. Durante as próximas aulas você terá a chance de vivenciar essa mudança de paradigma, mas já convido você a pausar por um instante

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

e pensar em exemplos do seu próprio contexto profissional em que esse contraste já apareceu.

Falando em ferramentas, é impossível tratar de IaC sem mencionar o **Terraform**. Criado pela HashiCorp, ele se consolidou como padrão de mercado por oferecer um modelo declarativo claro e um ecossistema de provedores que cobre praticamente todos os principais serviços de nuvem. Com ele, você descreve o estado desejado da sua infraestrutura e a ferramenta se encarrega de calcular o que precisa ser criado, alterado ou destruído para chegar lá. Essa simplicidade de raciocínio — declarar o que você quer, e não o passo a passo de como chegar até lá — é um dos grandes diferenciais em relação a scripts manuais. Ao final dessa aula, veremos como até mesmo um arquivo com poucas linhas já é capaz de criar um recurso funcional, mas aqui você já pode perceber a lógica por trás do conceito: o código se torna a única fonte de verdade da sua infraestrutura.

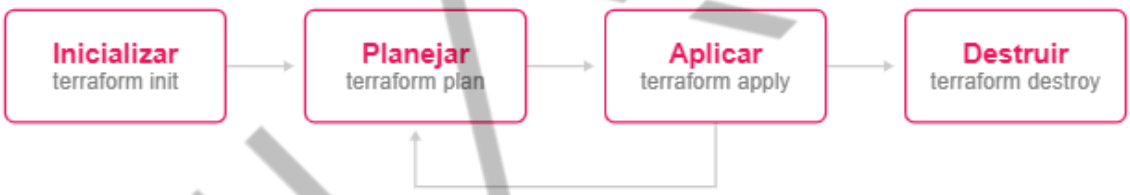


Figura 1 - Ciclo de vida Terraform
Fonte: Elaborado pelo autor (2025)

É nesse contexto que também surge o **OpenTofu**, um fork do Terraform mantido pela comunidade e sustentado por uma governança aberta. Em 2023, em um movimento da HashiCorp de alterar a licença do Terraform, gerou-se muitas preocupações sobre a continuidade do projeto como software livre. A comunidade, então, respondeu rapidamente, criando o OpenTofu como alternativa compatível e com o compromisso de manter o projeto aberto.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com



Figura 2 - Histórico OpenTofu
Fonte: Elaborado pelo autor (2025)

Na prática, isso significa que você pode utilizar tanto Terraform quanto OpenTofu sem grandes diferenças de uso, mas é importante entender as implicações estratégicas: empresas que dependem de transparência e de ecossistemas realmente livres tendem a preferir o OpenTofu.

Seguindo, vamos explorar a **estrutura básica de um projeto IaC**. Normalmente, ele começa com um arquivo `main.tf`, onde os recursos principais são declarados. Podemos adicionar `variables.tf` para parametrizar valores, `outputs.tf` para expor informações úteis e `terraform.tfvars` para configurar ambientes específicos.

```
├── meu-projeto-tf/
│   └── main.tf
│   └── variables.tf
│   └── outputs.tf
│   └── terraform.tfvars
```

Essa separação não é obrigatória, mas se tornou uma convenção amplamente aceita porque ajuda a manter o código limpo e fácil de entender. Nas aulas, você terá a chance de criar esse esqueleto e começar a construir seu primeiro projeto. Essa

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

estrutura inicial é a base que sustentará tudo que faremos ao longo da disciplina. Quanto antes você se familiarizar com ela, mais natural será evoluir para projetos complexos envolvendo múltiplos módulos, automação em pipelines e integração com ferramentas de segurança.

Vamos detalhar um pouco a construção desses arquivos. Este conteúdo pode ser encontrado também no repositório GitHub que estará disponível com todos os arquivos das aulas para clone. Neste exemplo, estamos provisionando uma EC2

`main.tf`: onde definiremos a configuração do nosso recurso EC2.

```
    # Configura o provedor da AWS, especificando a região onde
os recursos serão criados.
    provider "aws" {
      region = "us-east-1" # Você pode alterar esta região
conforme sua necessidade.
    }

    # Define o recurso de instância EC2.
    resource "aws_instance" "exemplo_ec2" {
      ami           = "ami-0c55b159cbfafe1f0" # ID de uma AMI
(Amazon Machine Image) para Linux (exemplo: Ubuntu). Procure a
AMI mais recente para a sua região.
      instance_type = "t2.micro"           # Tipo da instância
(t2.micro é geralmente gratuito para começar).

      tags = {
        Name = "ExemploEC2Terraform" # Um nome descritivo para
a sua instância.
      }
    }
```

Exemplo main.tf
Fonte: Elaborado pelo autor (2025)

`variables.tf`: este arquivo serve para declarar as variáveis que usamos no nosso código. Isso nos permite reutilizar o código e alterá-lo facilmente sem ter que mexer diretamente no `main.tf`..

```
    # Declara a variável para a região da AWS.
    variable "aws_region" {
      description = "A região da AWS onde os recursos serão
criados."
      type        = string
      default     = "us-east-1" # Valor padrão caso não seja
especificado.
    }
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
    # Declara a variável para o tipo da instância EC2.
    variable "instance_type" {
      description = "O tipo da instância EC2 a ser criada (ex:
t2.micro, t3.small)."
      type        = string
      default     = "t2.micro"
    }

    # Declara a variável para o nome da instância EC2.
    variable "instance_name" {
      description = "O nome a ser dado à instância EC2."
      type        = string
      default     = "ExemploEC2Terraform"
    }

    # Declara a variável para o ID da AMI.
    variable "ami_id" {
      description = "O ID da AMI (Amazon Machine Image) a ser
usada para a instância."
      type        = string
      # Não definimos um default aqui, pois o ID da AMI varia
muito por região e pode mudar.
      # O usuário deverá fornecer este valor.
    }
```

Exemplo variables.tf
Fonte: Elaborado pelo autor (2025)

outputs.tf: após o Terraform criar os recursos, é útil saber algumas informações sobre eles. Este arquivo define o que queremos que o Terraform nos mostre.

```
    # Define a saída para o ID da instância EC2 criada.
    output "instance_id" {
      description = "O ID da instância EC2 recém-criada."
      value       = aws_instance.exemplo_ec2.id # Acessa o ID
do recurso aws_instance.exemplo_ec2.
    }

    # Define a saída para o IP público da instância EC2.
    output "public_ip" {
      description = "O endereço IP público da instância EC2."
      value       = aws_instance.exemplo_ec2.public_ip # Acessa
o IP público do recurso.
    }

    # Define a saída para o IP privado da instância EC2.
    output "private_ip" {
      description = "O endereço IP privado da instância EC2."
```

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

```
      value      = aws_instance.exemplo_ec2.private_ip # Acessa
o IP privado do recurso.
    }
```

Exemplo variables.tf
Fonte: Elaborado pelo autor (2025)

`terraform.tfvars`: este arquivo é onde você vai "preencher" os valores para as variáveis que declaramos em `variables.tf`.

```
    # Define os valores para as variáveis declaradas em
variables.tf.

    # Exemplo de um ID de AMI para Ubuntu 22.04 LTS na região
us-east-1.
    # Lembre-se de verificar e usar o ID correto para a sua
região!
    ami_id = "ami-053b0d53c279acc90" # Este é um exemplo,
procure o atual na console AWS!

    # Você pode sobrescrever os valores padrão se desejar, por
exemplo:
    aws_region = "us-west-2"
    instance_type = "t3.small"
    instance_name = "MinhaInstanciaTerraform"
```

Exemplo variables.tf
Fonte: Elaborado autor (2025)

Para fixar esse conhecimento, sugiro que você volte às aulas sempre que sentir necessidade de rever um conceito. Não hesite em pausar, testar e até errar durante os exercícios práticos, pois é justamente nesse processo que o aprendizado se consolida. Um bom exercício neste momento é refletir sobre os benefícios que a IaC pode trazer ao seu contexto de trabalho. Pergunte-se: quais problemas da sua rotina poderiam ser evitados se a infraestrutura fosse tratada como código? Compartilhe suas percepções com colegas ou anote em seu repositório pessoal, pois isso ajudará a conectar teoria e prática de forma mais profunda.

Benefícios do uso de IaC:

- Rapidez → Provisionamento em minutos.
- Consistência → Mesma infra em todos os ambientes.
- Segurança → Histórico auditável.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

- Colaboração → Equipes trabalhando em repositórios Git.

Lembre-se: este é apenas o começo. O verdadeiro poder da IaC será revelado conforme avançarmos para variáveis, outputs, estados remotos, módulos e automações completas com pipelines de CI/CD. Mas não subestime a importância desta primeira aula. É aqui que você internaliza os conceitos que farão toda a diferença quando estiver diante de um projeto de alta complexidade. Volte sempre que precisar, assista novamente às aulas, revise os comandos de instalação e releia esta seção. Quanto mais sólida for sua base, mais tranquilo será seu caminho rumo ao domínio completo da Infraestrutura como Código.



Figura 3 - Fluxo cultural IaC
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

MERCADO, CASES E TENDÊNCIAS

A adoção de **Infraestrutura como Código** deixou de ser uma aposta e se tornou padrão em grandes empresas e startups de tecnologia. Bancos, e-commerces e órgãos públicos já adotam IaC como parte de suas estratégias de **Cloud Computing e DevOps**. A principal motivação é a previsibilidade: em mercados altamente regulados, como o financeiro, poder provar que sua infraestrutura é rastreável e reproduzível é uma vantagem competitiva. Em startups de crescimento acelerado, a IaC viabiliza criar ambientes em minutos, permitindo que equipes foquem na entrega de produto em vez de perder tempo com configuração manual.

Um exemplo interessante vem da **Airbnb**, que compartilhou sua experiência ao construir um sistema de **Continuous Integration flexível para iOS**. O desafio era grande: centenas de engenheiros precisavam rodar builds e testes de forma paralela, sem que o processo se tornasse gargalo. A solução foi criar uma camada de automação que, inspirada nos princípios de IaC, provisionava e gerenciava dinamicamente os recursos de infraestrutura usados no pipeline. Com isso, a empresa conseguiu aumentar a eficiência, reduzir custos e melhorar a experiência dos desenvolvedores, mostrando que os conceitos de automação e infraestrutura declarativa vão muito além da nuvem e podem ser aplicados até mesmo em ecossistemas específicos como o mobile.

Outro case relevante é o da **Dell Technologies**, que documentou sua jornada rumo à IaC em grande escala. A empresa enfrentava o desafio de modernizar data centers e operações de TI que ainda dependiam de processos manuais e lentos. Ao adotar IaC como pilar estratégico, a Dell conseguiu padronizar provisionamento, acelerar a entrega de serviços e reduzir a complexidade de ambientes híbridos que combinam nuvem pública e infraestrutura on-premises. O relato destaca que IaC não é apenas ferramenta, mas parte essencial de uma transformação digital que permite alinhar TI às demandas do negócio. Esse exemplo mostra que empresas tradicionais e com enorme legado tecnológico podem se beneficiar da mudança de paradigma trazida pela IaC.

Se você quiser se aprofundar no tema, recomendo duas leituras complementares. O livro **“Infrastructure as Code: Dynamic Systems for the Cloud**

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Age" de Kief Morris (O'Reilly, 3ª edição, 2025), considerado referência para compreender os fundamentos e a filosofia por trás da IaC, e o livro **"Terraform: Up and Running: Writing Infrastructure as Code" de Yevgeniy Brikman (O'Reilly, 3ª edição, 2022)**, que é prático e direto ao ponto, focado especificamente no uso do Terraform em ambientes reais. Juntos, eles formam uma base teórica e prática indispensável para quem deseja dominar a área.

Temos ainda um artigo sobre o tema: "AIRBNB. Flexible Continuous Integration for iOS. Medium, 2023. Disponível aqui.

Além disso, sugiro acompanhar o canal oficial da **HashiCorp no YouTube**, que publica webcasts e demonstrações práticas sobre Terraform.

"Playlist canal oficial Hashcorp" – YouTube.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nessa aula, aprendemos conceitualmente o que é Infraestrutura como Código e suas diferenças em comparação ao modelo tradicional de gestão de Infraestrutura. Conhecemos o projeto do Terraform e seu fork, o OpenTofu, e falamos também sobre a estrutura do código e as vantagens que a implementação de IaC pode trazer aos times que usam DevOps no seu dia a dia, reforçando que IaC não é apenas ferramenta, mas parte de uma mudança cultural.

Ao final, provisionamos o seu primeiro recurso via código, uma máquina EC2 na AWS.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

REFERÊNCIAS

AIRBNB. **Flexible Continuous Integration for iOS**. Medium, 2023. Disponível em: https://medium.com/airbnb-engineering/flexible-continuous-integration-for-ios-4ab33ea4072f. Acesso em: 21 out. 2025.

DELL. **Journey to Infrastructure as Code**. 2023 Disponível em: https://infohub.delltechnologies.com/pt-br/p/journey-to-infrastructure-as-code/. Acesso em: 21 out. 2025.

GITHUB. **get.opentofu.org.** 2024. Disponível em: https://github.com/opentofu/get.opentofu.org. Acesso em: 21 out. 2025.

HASHICORP. **HashiCorp YouTube.** 2025. Disponível em: https://www.youtube.com/c/HashiCorp. Acesso em: 21 out. 2025.

HASHICORP. **Install Terraform**. 2024. Disponível em: https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli. Acesso em: 21 out. 2025.

OPENTOFU. **Installing OpenTofu**. 2024. Disponível em: https://opentofu.org/docs/intro/install/. Acesso em: 21 out. 2025.

OPENTOFU. **Standalone Installer**. 2024. Disponível em: https://opentofu.org/docs/intro/install/standalone/. Acesso em: 21 out. 2025.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

PALAVRAS-CHAVE

Infraestrutura. Automação.Cloud Computing.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com



PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com